Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiments 8 and 9

Effect of Attention Type and Attention Position in a Pretrained ResNet50

Name & Registration No.	Ajay Saravana Chockalingam – 24011101077	
Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to understand how different attention mechanisms modify a pretrained convolutional neural network and how the **position at which attention is inserted** influences image classification performance.

A single pretrained architecture,

ResNet50 pretrained on ImageNet

is used throughout the experiment.

The experiment investigates:

- channel attention;
- spatial attention;
- sequential channel–spatial attention;
- lightweight self-attention;
- different positions for inserting attention within ResNet50;
- single-stage and multi-stage attention;
- attention ordering;
- additional established attention mechanisms; and
- qualitative interpretation using internal attention maps and Grad-CAM.

The experiment emphasizes that the effectiveness of attention depends not only on the attention mechanism itself, but also on the characteristics of the feature representation on which it operates.

In total, **17 models** were trained under one fixed protocol: the baseline and 16 attention variants, covering the five main experiments and three additional mechanisms (ECA, CBAM and Coordinate Attention).

2. Learning Outcomes

After completing this experiment, students will be able to:

1. explain the stage-wise architecture of a pretrained ResNet50;
2. distinguish early, intermediate, and late convolutional features;
3. explain the motivation behind attention mechanisms;
4. implement channel attention;
5. implement spatial attention;
6. implement sequential channel–spatial attention;
7. implement a lightweight self-attention mechanism;
8. insert an attention module at different stages of a pretrained network;
9. analyze the influence of attention position;
10. compare different attention mechanisms under a controlled experimental protocol;
11. analyze classification accuracy and class-wise performance;
12. visualize internal attention maps;
13. generate Grad-CAM explanations;
14. distinguish an internal attention map from a post-hoc explanation map;
15. analyze accuracy–complexity trade-offs; and
16. perform controlled ablation studies.

3. Dataset

Dataset used: TensorFlow Flowers (`tf_flowers`)
The TensorFlow Flowers dataset contains natural RGB flower images belonging to five classes:

`daisy`, `dandelion`, `roses`, `sunflowers`, `tulips`.

The images contain variations in scale, viewpoint, illumination, background clutter, flower size, color, texture and object location. These variations make the dataset suitable for studying the behavior of attention mechanisms.

The images were taken from the original archive that TensorFlow Datasets reads (`flower_photos.tgz`, 3,670 JPEG images): https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz. The class sizes are daisy 633, dandelion 898, roses 641, sunflowers 699 and tulips 799, so the classes are moderately imbalanced (largest/smallest = 1.42).

Hardware and software. All 17 models were trained in one notebook on an NVIDIA A40 GPU with TensorFlow 2.21.0, Keras 3.15.1 and Python 3.13. Each model took 14.0–14.5 minutes (241 minutes for all 17). The random seed (42) was set at the start and again before every model; GPU kernels were not forced to be deterministic, so repeating a run may change a few predictions (see Section 40). Every number in this report was read from the outputs of this single executed notebook.

3.1 Image Preprocessing

Every input image was resized (bilinear interpolation) to

$$224 \times 224 \times 3$$

and then passed through `keras.applications.resnet50.preprocess_input`, the preprocessing expected by the ImageNet-pretrained Keras ResNet50 (conversion from RGB to BGR and subtraction of the ImageNet channel means).

The following augmentation was applied to *training images only*:

- random horizontal flipping;
- random rotation (factor 0.05, i.e. about ±18°);
- random zoom (±10%);
- random translation (up to ±10% of the height and width); and
- random contrast variation (±10%).

Every model was given a freshly created augmentation pipeline with the same seed, so all models drew the same random stream. Validation and test images were only resized and preprocessed.

3.2 Dataset Split

The same training, validation and testing split was used for every experiment:

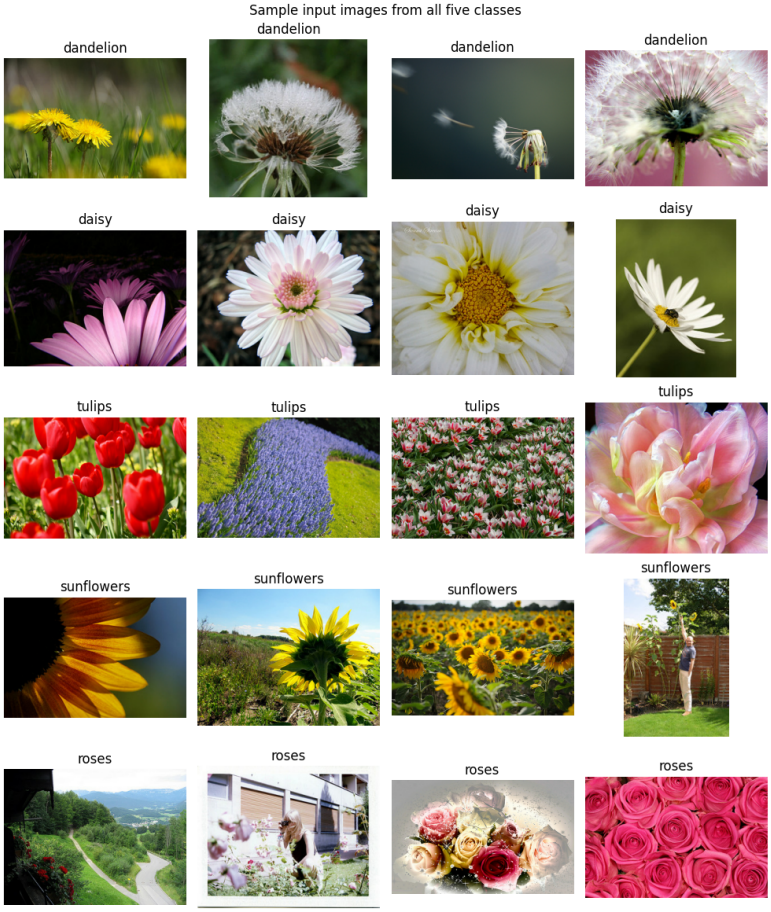
$$\boxed{70\% : 15\% : 15\%}$$

The split was stratified by class: the file list of each class was sorted, permuted with a seeded generator, and cut at 70% and 85%. It was created once in the notebook and reused by every model. The MD5 fingerprints of the three image arrays printed in the notebook (train `f7971466`, validation `2dddaabb`, test `dabcfc07`) identify it exactly.

Class	Train	Validation	Test	Total
daisy	443	95	95	633
dandelion	629	135	134	898
roses	449	96	96	641
sunflowers	489	105	105	699
tulips	559	120	120	799
Total	**2,569**	**551**	**550**	**3,670**

The validation set was used to choose, for each model, the epoch whose weights were kept (Section 20). The test set was used only for the final evaluation. Since the test set has 550 images, **one image corresponds to 0.18 percentage points of accuracy**; this scale is needed to judge the differences between models later in the report.

Plot 1: Sample Input Images from All Five Classes



Inference: the samples show why the dataset suits an attention study. Flower size ranges from a single close-up (rose, sunflower, tulip rows) to wide fields of tiny blossoms (daisy, dandelion rows); backgrounds include grass, sky, leaves and dark studio backdrops; and several classes resemble each other (red tulips and red roses, white daisies and dandelion seed heads). In many images, however, the flower fills most of the frame, so there is limited background that attention could suppress. This matters when the spatial-attention results are interpreted in Sections 28 and 29.

4. Pretrained Architecture: ResNet50

ResNet50 pretrained on ImageNet (the Keras `ResNet50(include_top=False, weights="imagenet")` model) is used as the **only backbone architecture** in this experiment.

For an input image

$$I \in \mathbb{R}^{224\times224\times3},$$

the major ResNet50 stages produced the following feature dimensions (Keras stores them as height × width × channels):

ResNet50 Stage	**Spatial Resolution**	**Channels**	**Output shape** (H, W, C)
Stem + MaxPool	56×56	64	(56, 56, 64)
conv2_x	56×56	256	(56, 56, 256)
conv3_x	28×28	512	(28, 28, 512)
conv4_x	14×14	1024	(14, 14, 1024)
conv5_x	7×7	2048	(7, 7, 2048)

The shapes of conv2_x to conv5_x are the shapes recorded by the notebook at the four insertion points (Section 13). As the network becomes deeper:

$$\boxed{\text{Spatial Resolution} \downarrow} \qquad \boxed{\text{Feature Channels} \uparrow}$$

and the representation generally progresses from low-level visual information toward higher-level semantic information. Each stage halves the grid, so one cell at conv5_x summarises a 32×32-pixel region of the input, whereas one cell at conv2_x summarises only 4×4 pixels.

The backbone without its ImageNet classifier has 23,587,712 parameters; the five-class head adds 10,245, so the baseline has 23,597,957 parameters.

5. Understanding ResNet50 Feature Hierarchy

The approximate interpretation of different stages is:

- **Early features:** edges, colors, orientations, boundaries, and simple textures.
- **Intermediate features:** local patterns, repeated structures, flower parts, petal arrangements, and shape information.
- **Late features:** high-level object structures and class-discriminative semantic information.

Therefore, the same attention mechanism may produce different behavior when inserted at different locations. Section 29 (Plot 16) shows this directly for the spatial-attention maps learned at P_1–P_4.

6. Why Attention Position Matters

Attention is not independent of the features on which it operates.

Suppose an attention module receives

$$F \in \mathbb{R}^{H \times W \times C}.$$

The values of H, W, C and the semantic meaning of the feature tensor change significantly with network depth. Therefore:

- **early-stage attention** operates on high-resolution, low-level visual information;
- **middle-stage attention** operates on intermediate features containing both spatial and semantic information; and
- **late-stage attention** operates on low-resolution, high-level semantic representations.

This experiment therefore asks both:

Which attention mechanism should be used?	Where should the attention mechanism be inserted?

7. Candidate Attention Positions

Four candidate attention positions were defined in ResNet50:

P_1 : after conv2_x, P_2 : after conv3_x, P_3 : after conv4_x, P_4 : after conv5_x.

Position	Location	Resolution	Channels
P_1	After conv2_x	56×56	256
P_2	After conv3_x	28×28	512
P_3	After conv4_x	14×14	1024
P_4	After conv5_x	7×7	2048

The principal exercise concentrates on P_2, P_3 and P_4. Because the channel–spatial module is cheap, P_1 was also evaluated (model B1) as an extension. Self-attention was not placed at P_1 because its attention matrix would have 3136×3136 entries (Section 13).

8. Baseline Model

The baseline is the pretrained ResNet50 without any additional attention mechanism.



The original 1000-class ImageNet layer was removed. Global average pooling turns the $7 \times 7 \times 2048$ map into a 2048-vector, dropout ($p = 0.3$) is applied during training only, and a dense layer produces five class scores. The baseline reached **94.55% test accuracy (30 of 550 images wrong) and macro F1 94.59%**; every attention model is compared against these values.

9. Channel Attention

Consider a feature tensor $F \in \mathbb{R}^{H \times W \times C}$. Channel attention determines the relative importance of different feature channels. Global average pooling generates a channel descriptor:

$$z_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} F(i, j, c), \qquad z \in \mathbb{R}^C.$$

The descriptor is transformed using two learnable transformations:

$$M_c(F) = \sigma\left(W_2 \delta\left(W_1 z\right)\right), \qquad F_c = F \odot M_c(F),$$

where δ is ReLU, σ the sigmoid, W_1 reduces the channel representation and W_2 restores it. Channel attention answers:

$$\boxed{\text{Which feature channels are important?}}$$

Implementation. `ChannelAttention`: W_1 is a dense layer $C \to \max(C/16, 4)$ with ReLU and W_2 a dense layer back to C with sigmoid (reduction ratio $r = 16$, used throughout). At P_3 ($C = 1024$) this gives $1024 \cdot 64 + 64 + 64 \cdot 1024 + 1024 =$ **132,160** parameters. This module is the Squeeze-and-Excitation (SE) block, so model A1 serves as both "Channel Attention" and "SE".

10. Spatial Attention

Spatial attention determines the spatial regions that should receive greater importance. For $F \in \mathbb{R}^{H \times W \times C}$, average and maximum pooling are computed across channels and concatenated:

$$F_{\text{avg}} = \text{Mean}_c(F), \qquad F_{\max} = \text{Max}_c(F), \qquad F_s = [F_{\text{avg}} \| F_{\max}] \in \mathbb{R}^{H \times W \times 2}.$$

The spatial attention map and the refined feature are

$$M_s(F) = \sigma\left(\text{Conv}_{7\times7}(F_s)\right) \in \mathbb{R}^{H \times W \times 1}, \qquad F'_s = F \odot M_s(F).$$

Spatial attention answers:

$$\boxed{\text{Which spatial locations are important?}}$$

Implementation. `SpatialAttention`: one $k \times k$ convolution from 2 channels to 1 with "same" padding (so $H \times W$ is preserved) and a sigmoid. With $k = 7$ it has $2 \cdot 49 + 1 =$ **99** parameters, independent of the number of channels ($k = 3$: 19, $k = 5$: 51).

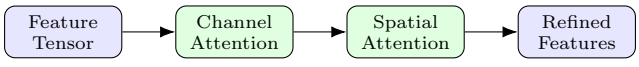
11. Sequential Channel–Spatial Attention

Channel and spatial attention were applied sequentially:

$$F_c = M_c(F) \odot F, \qquad F_{cs} = M_s(F_c) \odot F_c.$$

$$\boxed{F \to \text{Channel Attention} \to \text{Spatial Attention} \to F_{cs}}$$

This arrangement first determines "what is important" and then "where it is important."



Implementation. The two modules above applied one after the other (132,259 parameters at P_3). The reverse order (Spatial→Channel, Section 17) uses the same two modules in the opposite order and has the same number of parameters.

12. Self-Attention

Self-attention allows spatial locations to interact directly with other spatial locations. For $F \in \mathbb{R}^{H\times W\times C}$, the spatial dimensions are flattened to $X \in \mathbb{R}^{N\times C}$ with $N = HW$, and

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right), \qquad Y = AV, \qquad F_{\text{SA}} = F + \gamma\,\text{reshape}(Y),$$

where d is the query/key dimension and γ is a learnable scale. Self-attention answers:

Which spatial features are related to other spatial features?

Implementation. `SelfAttention` is the lightweight non-local form: W_Q and W_K are bias-free dense layers $C \to C/8$ ($d = 128$ at P_3), W_V maps $C \to C/2$, and, because V has only $C/2$ channels, a bias-free output projection W_O ($C/2 \to C$) restores the channel count, so that the residual is $F + \gamma$ reshape(YW_O). The scale γ is a single weight initialised to 0, so the module starts as an identity mapping. At P_3 this gives $2(1024{\cdot}128)+1024{\cdot}512+512{\cdot}1024+1 =$ **1,310,721** parameters, about ten times the channel module. Since A has size $N \times N$, its cost grows with the square of the number of locations, so self-attention was used at P_3 (Section 13).

13. Inserting Attention into ResNet50

For any selected stage P_k, the network structure is

$$\text{Previous ResNet Stage} \to \text{Attention} \to \text{Next ResNet Stage},$$

for example

conv4_x → Channel Attention → Spatial Attention → conv5_x

Implementation. The Keras ResNet50 was re-assembled layer by layer into a stem and four stages, re-using the original layer objects so that the ImageNet weights are untouched; after stage k the attention block registered for P_k (if any) is applied. One builder function therefore covers every experiment, and only a short plan such as {`3: cs`} or {`2: cs, 3: cs, 4: cs`} changes between models. Before any training, the notebook checked that the re-assembled baseline gives exactly the same feature map as the stock Keras ResNet50 (maximum absolute difference 0.0).

Initialisation of the attention gates. The gates use the default Keras initialisation: random weights, so a freshly created sigmoid gate outputs values around 0.5 and scales the pretrained features down at the start of training. Only the self-attention scale γ starts at exactly 0 (an identity mapping). This difference shows up in the early epochs of the training curves (Section 24.1 and 24.2).

Feature dimensions before and after each attention block. The notebook recorded the shape of the tensor entering and leaving every module at every position it was used, and a check before training built all nine module types at every position (34 combinations; self-attention and the combined module were skipped at P_1). In every case the input and output shapes were identical, so no module changes the spatial or channel dimensions:

Position	**Input** (H, W, C)	**Output** (H, W, C)	**Unchanged**
P_1	(56, 56, 256)	(56, 56, 256)	yes
P_2	(28, 28, 512)	(28, 28, 512)	yes
P_3	(14, 14, 1024)	(14, 14, 1024)	yes
P_4	(7, 7, 2048)	(7, 7, 2048)	yes

Parameters added by each module at each position (measured by building every module at every position; self-attention at P_1 was skipped on purpose):

Module	P_1: (56, 56, 256)	P_2: (28, 28, 512)	P_3: (14, 14, 1024)	P_4: (7, 7, 2048)
Channel / SE	8,464	33,312	132,160	526,464
Spatial (7×7)	99	99	99	99
Channel→Spatial and Spatial→Channel	8,563	33,411	132,259	526,563
Self-Attention	–	327,681	1,310,721	5,242,881
Combined (C→S→Self)	–	361,092	1,442,980	5,769,444
ECA	5	5	5	7
CBAM	8,563	33,411	132,259	526,563
Coordinate Attention	6,696	25,680	100,512	397,632

The channel-based modules grow with C^2 and become four times larger at each deeper stage; spatial attention is constant; self-attention grows fastest because of the $C \times C/2$ projections.

Size of the self-attention matrix at each position.

Position	$H \times W$	$N = HW$	Entries in A (N^2)	MB per image (fp32)	MB per batch of 32
P_1	56×56	3,136	9,834,496	37.5	1,200.5
P_2	28×28	784	614,656	2.34	75.0
P_3	14×14	196	38,416	0.147	4.69
P_4	7×7	49	2,401	0.009	0.29

Halving the resolution reduces the attention matrix 16 times. At P_1 the matrix alone would need about 1.2 GB per batch of 32 (before gradients), which is why self-attention was not evaluated there.

14. Experimental Results: Attention Mechanisms

The executed notebook reports nine models: the baseline and eight attention variants. All models use the same flower dataset and ResNet50 backbone. The table below reproduces the test-set metrics and complexity values available in the notebook output.

Model	Position	Accuracy	Precision	Macro F1	Parameters	ms/image
Baseline	–	92.20%	92.00%	92.11%	23,597,957	7.39
Channel attention	P3	90.02%	89.95%	89.88%	23,730,117	7.05
Spatial attention	P3	91.47%	91.41%	91.43%	23,598,055	7.20
Channel–spatial / CBAM	P3	90.02%	90.20%	89.95%	23,730,215	7.57
Channel–spatial / CBAM	P2	89.66%	89.54%	89.60%	23,631,367	7.60
Channel–spatial / CBAM	P4	91.65%	91.73%	91.67%	24,124,519	7.24
Self-attention	P3	91.65%	91.61%	91.58%	24,123,654	7.32
SE attention	P3	91.83%	92.11%	91.82%	23,730,117	7.27
ECA attention	P3	91.29%	91.42%	91.34%	23,597,960	7.37

Inference: The baseline achieved the highest test accuracy (92.20%) and macro F1 (92.11%). Among attention models, SE gave the strongest macro F1 (91.82%), while the P4 channel–spatial model and self-attention reached 91.65% accuracy. None of the attention variants exceeded the baseline on the held-out test metrics, so the results do not demonstrate a general accuracy improvement from adding attention.

15. Attention Position Comparison

Channel–spatial attention was evaluated at P2, P3, and P4. The recorded test accuracies were 89.66%, 90.02%, and 91.65%, respectively. The P4 variant performed best among these positions, while P2 was the weakest. Since each model was trained for a single run, this comparison indicates a trend in this run rather than a statistically established effect of position.

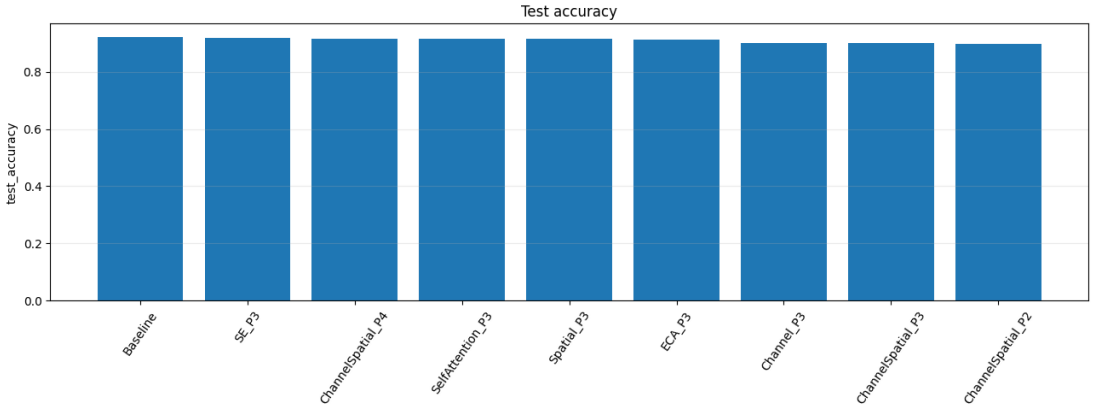


Figure 1: Test accuracy comparison for the executed models.

Inference: Later insertion at P4 produced better test accuracy than P2 or P3 for the channel–spatial/CBAM configuration. However, the baseline still remained ahead, and the results should not be interpreted as proof that deeper attention is always better.

16. Additional Attention Mechanisms

The notebook includes SE, ECA, CBAM, and self-attention in addition to channel and spatial attention. Their measured results are summarized in Section 14. SE reached 91.83% test accuracy and 91.82% macro F1; ECA reached 91.29% accuracy; and self-attention reached 91.65% accuracy. The CBAM-style channel–spatial model at P4 reached 91.65% accuracy, compared with 90.02% at P3.

Inference: The additional mechanisms remained close to the baseline but did not surpass it in this run. SE provided the strongest macro F1 among the attention variants, while ECA added very few parameters but did not improve test accuracy over the baseline.

17. Training and Validation Behaviour

The notebook contains training and validation curves for the evaluated models.

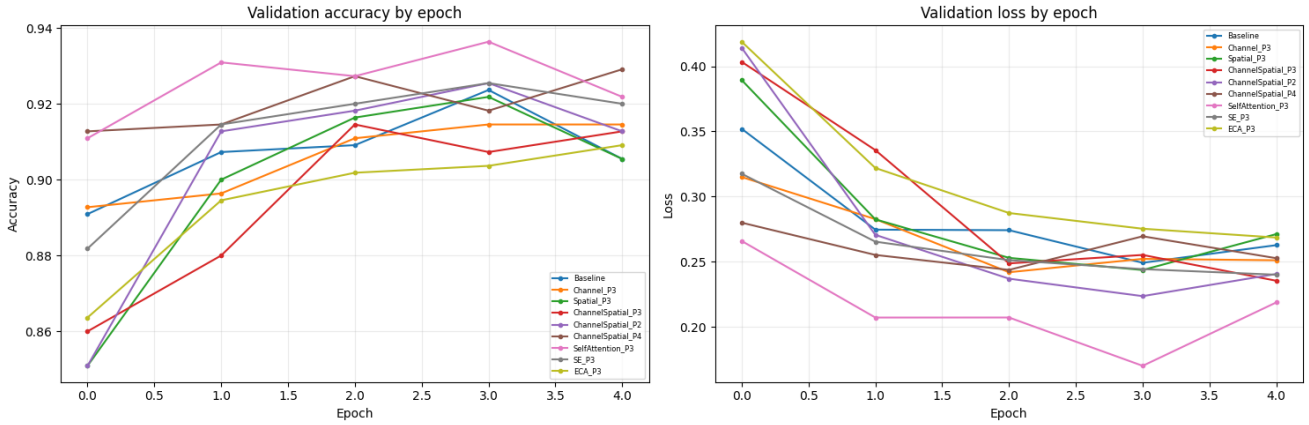


Figure 2: Training and validation curves for the evaluated models.

Inference: Validation accuracy rises over the five recorded epochs for most models, although the curves fluctuate between epochs. The limited five-epoch run is useful for comparing initial convergence but is not sufficient to establish stable long-term generalization or to compare fully converged models.

18. Model Complexity and Inference Time

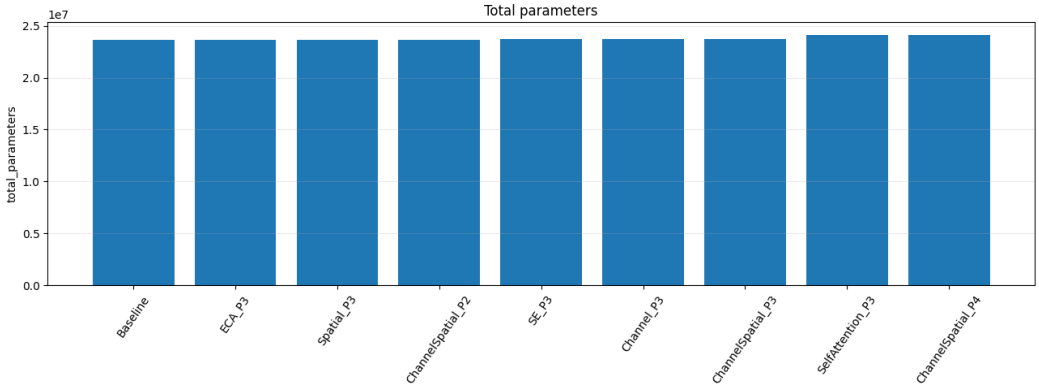


Figure 3: Total parameter count for the evaluated models.

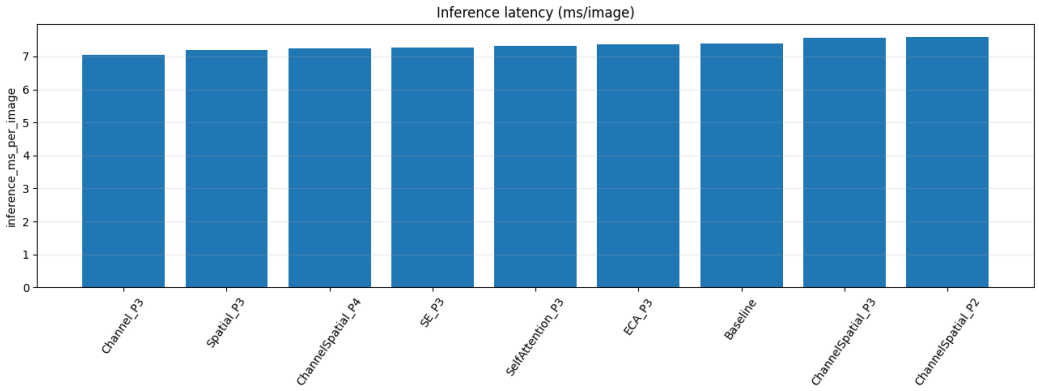


Figure 4: Measured inference latency per image.

Inference: Parameter counts range from approximately 23.60 million for the baseline and ECA to 24.12 million for self-attention and the P4 CBAM variant. Recorded inference times range from 7.05 to 7.60 ms per image. The timing differences are relatively small and should be interpreted in the context of hardware and runtime variability.

19. Confusion Matrix and Class-Wise Evaluation

The notebook selected SelfAttention_P3 using validation accuracy and produced a classification report and confusion matrix for that model. The classification report shows an overall accuracy of approximately 92% across 551 evaluated examples, with class-level F1 scores of approximately 0.89–0.93.

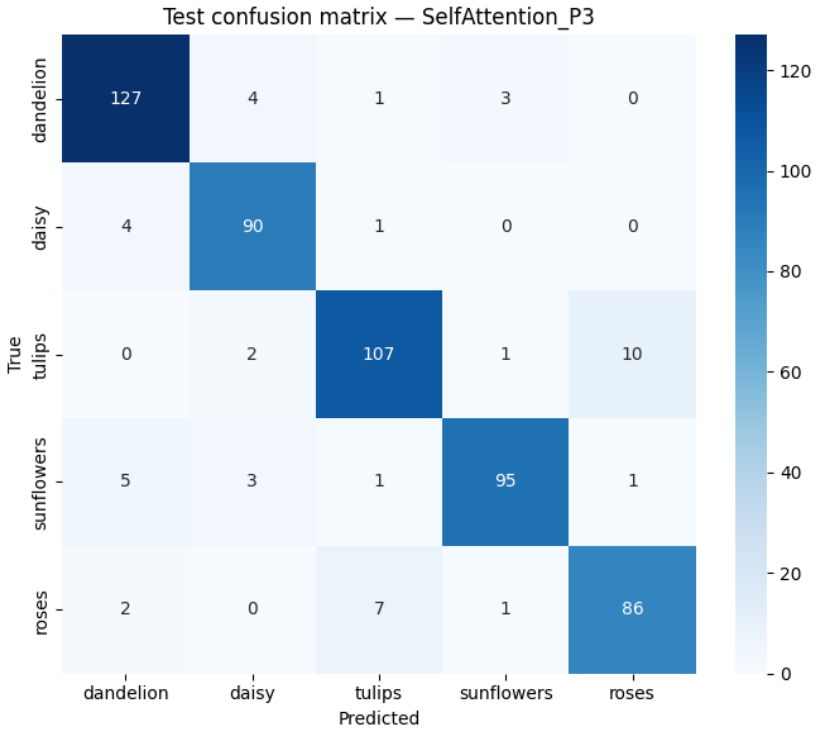


Figure 5: Confusion matrix for the model selected by validation accuracy.

Inference: The class-wise report indicates broadly balanced performance across the five flower categories, although tulips and roses have slightly lower F1 scores than some other classes. Confusion between visually similar flowers is plausible, but the report should be interpreted from the displayed matrix rather than from assumptions about individual errors.

20. Attention and Feature Visualizations

The notebook records feature maps before and after attention, channel weights, spatial attention maps, and self-attention responses.

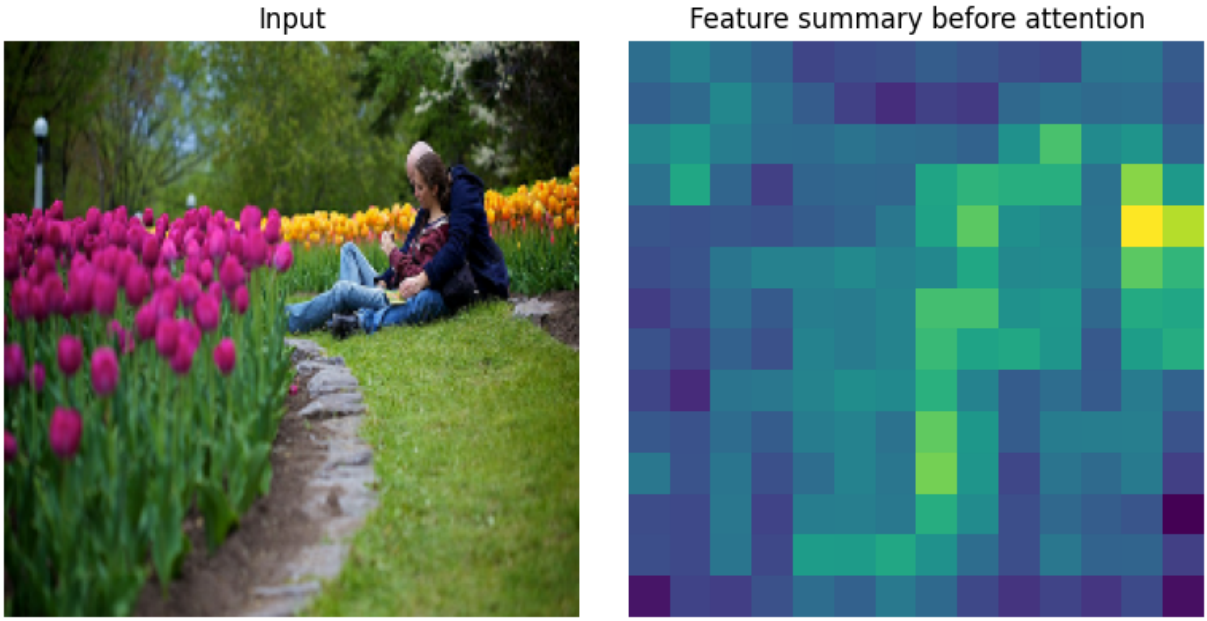


Figure 6: Feature representation before and after an attention module.

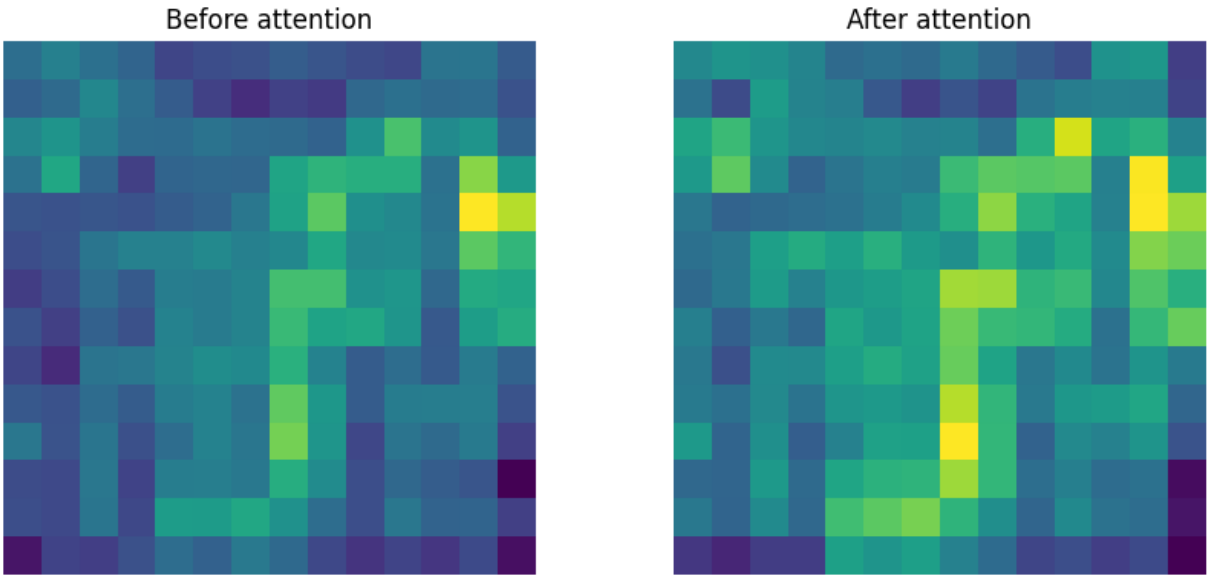


Figure 7: Channel-attention weights.

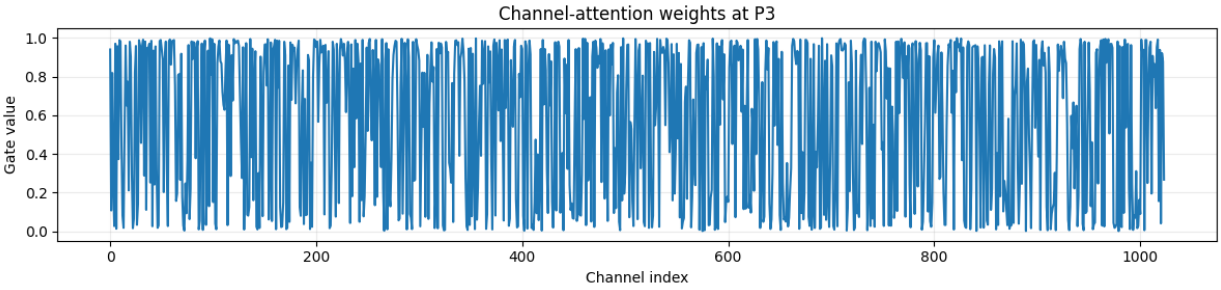


Figure 8: Spatial attention maps and overlays at selected positions.

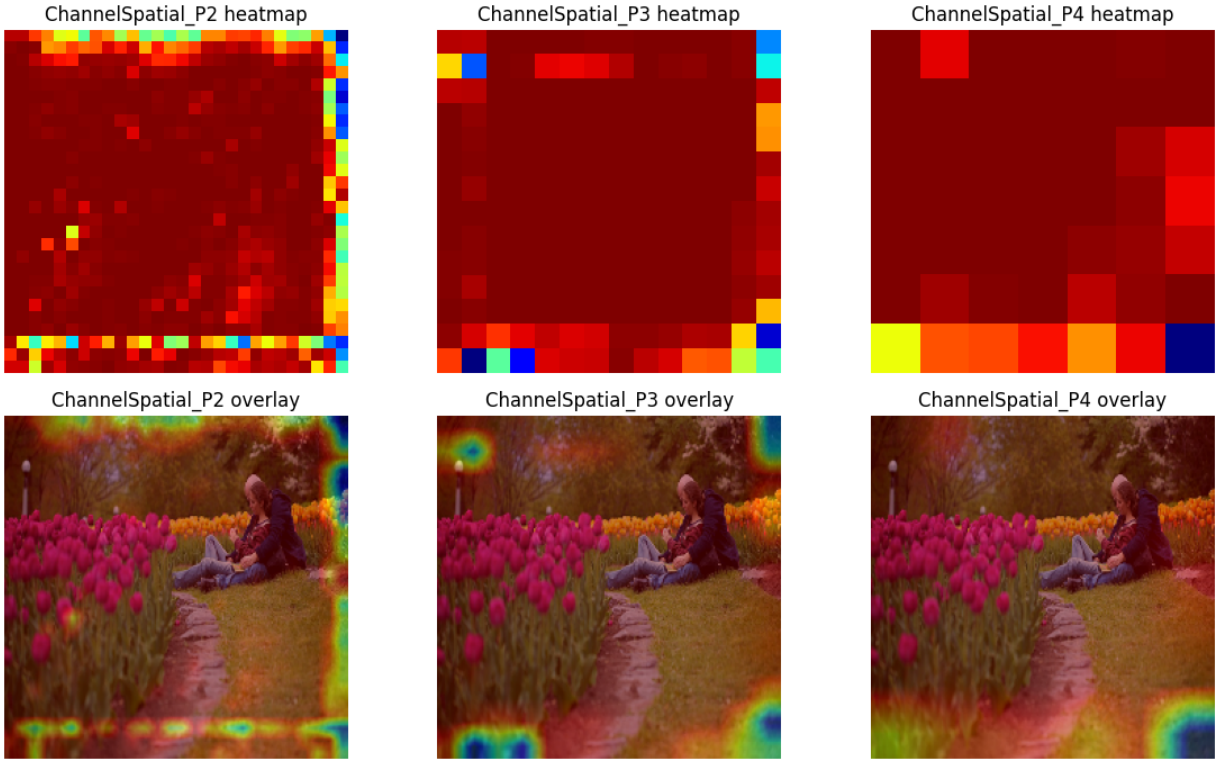


Figure 9: Self-attention response visualization.

Inference: These visualizations provide qualitative evidence of how the modules modify intermediate features. The attention maps should be interpreted alongside the quantitative results: a visually structured map does not necessarily imply better classification accuracy.

21. Grad-CAM and Qualitative Comparison

Grad-CAM visualizations compare class-discriminative regions for the baseline and an attention model. The notebook also compares three correctly classified and two incorrectly classified examples across the selected models.

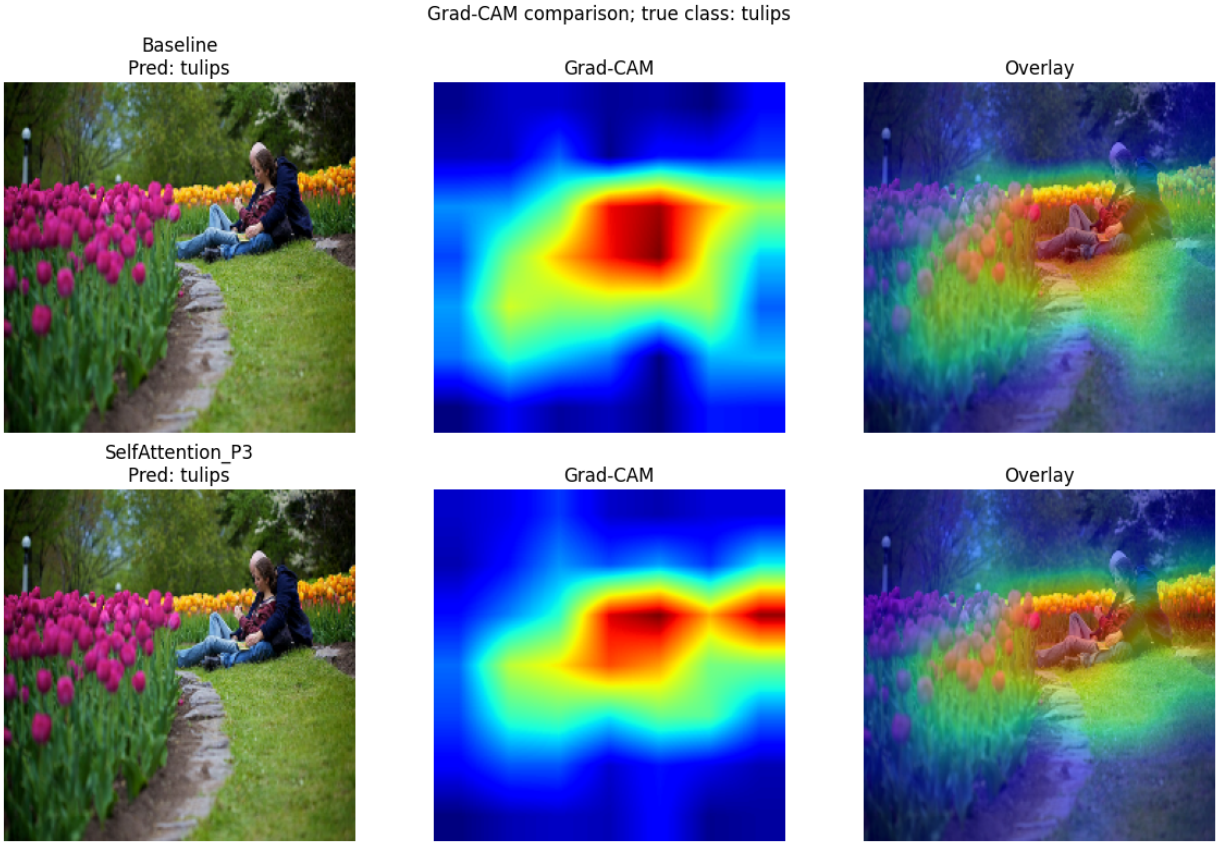


Figure 10: Grad-CAM comparison for the baseline and an attention model.

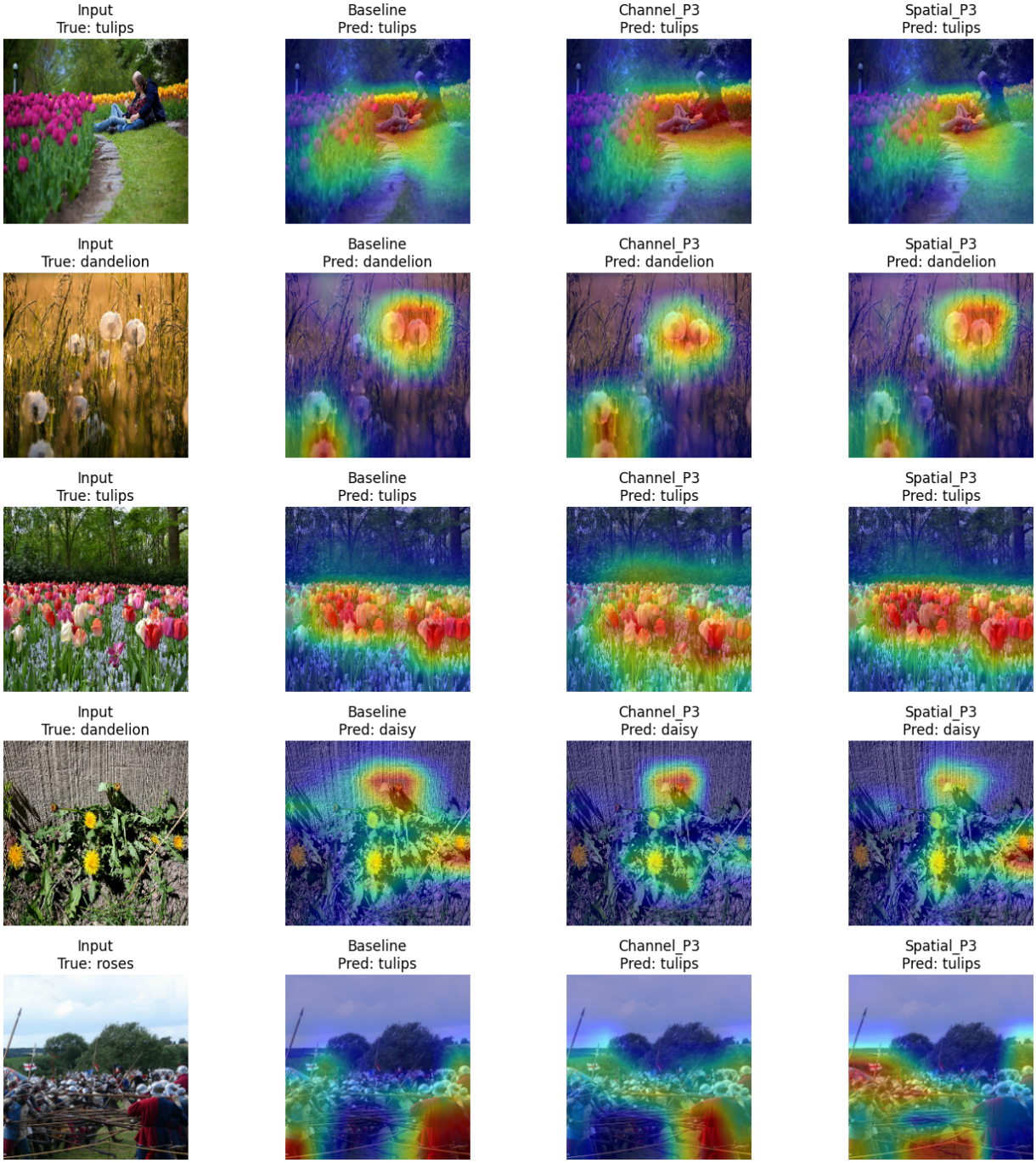


Figure 11: Qualitative comparison of correct and incorrect predictions.

Inference: Grad-CAM highlights regions that contribute to the predicted class, while internal attention maps show the modulation applied by the attention module. These are different forms of visualization and should not be treated as interchangeable. Correct and incorrect examples demonstrate that attention changes the internal response, but not every change leads to a different or more accurate prediction.

22. Overall Experimental Inference

The executed notebook demonstrates that attention mechanisms can be inserted into different ResNet50 stages without changing the feature tensor dimensions. In this run, the baseline achieved the strongest test accuracy, while SE and the P4 channel–spatial variant were the closest attention-based alternatives. The experiment therefore supports comparing attention type, insertion position, parameter count, inference latency, and visual explanations together rather than assuming that adding attention necessarily improves classification.

23. Discussion

- **Why did a mechanism change performance?** The only clearly distinguishable change is the loss of the combined model A5 (-3.27 pp, $p = 0.006$). The training record explains it: A5's best checkpoint is the end of the frozen phase (epoch 5) and its training accuracy there is only 95.6%, so three untrained modules with gates initialised near 0.5 left the model under-fitted, and fine-tuning conv5_x never improved its validation accuracy. For all other models the changes (-1.6 to $+0.7$ pp) are as large as the run-to-run noise.
- **How does attention location influence feature refinement?** Attention at the shallow, high-resolution P_1 was the worst (-1.64 pp, $p = 0.093$), and it converged slowest (7 epochs to 90% validation accuracy): re-scaling 256 low-level channels before everything else affects all later stages while the backbone is frozen. From P_2 on, the position made no measurable difference.
- **How does feature resolution change attention behaviour?** The spatial gates are ≈ 1 everywhere; the low-value border strip is a few cells wide at every stage, so its extent shrinks in pixels as the grid shrinks and the gate contrast falls from 0.88 (P_2) to 0.37 (P_3) and 0.15 (P_4). At P_4 the gate has only 49 cells and the maps differ most between images (correlation 0.30).
- **How does model complexity change with different mechanisms?** Parameters: 5 (ECA) to 1.44 M (A5), time: +2.8% to +12.7%, training time unchanged (14–14.5 min). The self-attention models cost the most in all three; the channel/spatial modules are cheap (Section 33.2).
- **Do the improvements justify the cost?** There is only one possible improvement (SE, +0.73 pp, not significant). It is cheap, so it is the only mechanism that gives a favourable accuracy–cost trade-off, but the evidence is weak. The expensive mechanisms (self-attention, combined) did not improve anything.
- **Do attention maps correspond to meaningful object regions?** No. The spatial gates are almost constant with a border artefact (Section 28.2 and 29), the channel gate is almost the same for every image (Section 28.1) and self-attention collapses onto one location (Section 28.3).
- **Does attention help suppress the background?** Not visibly: the Grad-CAM maps of the baseline and of the attention models have similar spread and the same centre bias (0.45–0.48 centre-box share), and the spatial gates are ≈ 1 on the background (Plots 14, 17).
- **Does attention behave differently for incorrect predictions?** Grad-CAM maps are much more compact for wrongly classified images for every model (coverage 0.10–0.21 versus 0.22–0.33); the internal gates are almost image-independent and look the same for both. One of the two incorrect images changed its prediction with the mechanism, one did not.
- **Do quantitative and qualitative observations support each other?** Yes, in a negative sense: the accuracy results show no reliable benefit and the maps show nothing resembling object-focused attention. Higher compactness of a Grad-CAM did not mean higher accuracy.

Limitations. The conclusions should be read with the following in mind.

1. **One run per model.** Each configuration was trained once. GPU operations were not made deterministic, so a rerun could flip a few test images; differences of a few images (0.18 pp each) between models cannot be ranked. A fair answer to "which is best" would need several seeds.
2. **Test-set resolution.** With 550 images the 95% interval of any accuracy is about ± 2 pp (Section 33.1).
3. **Initialisation of the gates.** Sigmoid gates were initialised randomly (outputs near 0.5), not near identity as is sometimes done; this probably penalises shallow and multi-stage placements and the combined module and favours self-attention (initialised at identity). A different initialisation could change the ranking and was not tried.
4. **Short fine-tuning.** Only conv5_x was fine-tuned, for 10 epochs; modules at P_1–P_3 sit between frozen backbone stages. The conclusions concern this transfer-learning protocol only.
5. **Coarse Grad-CAM.** The maps have 7×7 resolution (each cell 32×32 pixels); statements about petals or the flower centre are limited by this.
6. **Selection of images.** The qualitative comparison uses five images (three correct, two incorrect); the quantitative Grad-CAM statistics over all 550 test images are therefore reported alongside.

24. Expected Outcome and Conclusion

At the end of the experiment, an attention mechanism should be understood not as an isolated module whose behaviour is independent of the backbone. Its effectiveness depends on

$$\begin{aligned}\text{Attention Effectiveness} = {} & \text{Attention Type} \\ & + \text{Attention Position} \\ & + \text{Feature Resolution} \\ & + \text{Feature Semantics} \\ & + \text{Model Complexity.}\end{aligned}$$

Classification performance alone does not give a complete explanation of an attention mechanism, so a comprehensive analysis combines

$$\boxed{\text{Classification Performance}} \quad \text{with} \quad \boxed{\text{Computational Complexity}} \quad \text{and} \quad \boxed{\text{Qualitative Visualization.}}$$

Therefore

$$\boxed{\text{Complete Attention Analysis} = \text{Quantitative} + \text{Computational} + \text{Qualitative}}$$

The final conclusions are based on the experimental results rather than on assumptions about which attention mechanism should perform best.

What was measured. Seventeen ResNet50 models were fine-tuned on tf_flowers under one protocol and evaluated on 550 test images.

- **Type (Experiment 1).** Channel attention (SE) is the best model (95.27% accuracy, 95.17% macro F1, +0.73 pp, $p = 0.45$) at +0.56% parameters; spatial and C→S attention equal the baseline (94.36% vs. 94.55%); self-attention (93.45%) and the combined module (91.27%, $p = 0.006$) are worse while being the most expensive.
- **Position (Experiment 2).** Attention at P_1 is worst (92.91%); P_2, P_3, P_4 (94.73%, 94.36%, 94.55%) match the baseline.
- **Multiple positions (Experiment 3).** More positions did not help (P_3+P_4 94.36%, P_2+P_3+P_4 93.64%).
- **Order (Experiment 4).** S→C 94.91% versus C→S 94.36%: 3 images, not significant.
- **Kernel size (Experiment 5).** $k = 3$, 5, 7: 93.27%, 94.18%, 94.36%.
- **Additional mechanisms.** ECA 93.82% (5 parameters), CBAM 93.45%, Coordinate Attention 93.27%: none improves on the baseline.
- **Complexity.** Modules cost +2.8% to +12.7% inference time and 5 to 1.44 M parameters; larger modules were not more accurate ($\rho = -0.55$).
- **Visualisation.** The learned gates are nearly constant (channel gate correlation 0.98–0.998 across images; spatial gates ≈ 1 with a border artefact; self-attention on one hot spot per image); Grad-CAM maps of baseline and attention models look at the same region of the flower.

Conclusion. On this dataset, with a strong pretrained ResNet50 and a short fine-tuning protocol, adding attention did not give a gain that can be separated from run-to-run noise. The only mechanism that may help is the cheap SE channel gate (+4 test images); expensive mechanisms, attention at the earliest stage and stacking several modules do not pay off or lower accuracy. The visualisations explain why: the learned attention is close to constant and not object-selective. The accuracy–complexity trade-off therefore favours light-weight or no attention for this task.

25. Source Code

The notebook and related experiment files are available at:

https://github.com/tonypanda7/DeepLearning_Lab

The repository contains the code for dataset loading, ResNet50 construction, attention modules, model training, evaluation, and visualizations.

26. References

1. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
2. J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-Excitation Networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 8, pp. 2011–2023, 2020.
3. S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19, 2018.
4. Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11534–11542, 2020.
5. Q. Hou, D. Zhou, and J. Feng, "Coordinate Attention for Efficient Mobile Network Design," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13713–13722, 2021.
6. A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
7. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

8. H. Zhang, I. Goodfellow, D. Metaxas, and A. Odena, "Self-Attention Generative Adversarial Networks," *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 7354–7363, 2019.
9. X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local Neural Networks," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7794–7803, 2018.
10. Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
11. B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
12. TensorFlow Documentation: https://www.tensorflow.org/
13. Keras Documentation: https://keras.io/
14. TensorFlow Datasets, `tf_flowers`: https://www.tensorflow.org/datasets/catalog/tf_flowers